

ACADEMIC REPORT

Credit-Card Fraud Detection Using Machine Learning and Deep Learning

An end-to-end pipeline with LightGBM, a PyTorch neural network and SHAP explainability

Submitted by: Charan Reddy

Programme: MBA (Data Science & Data Analytics), 2025-27

Institution: Symbiosis Centre for Information Technology (SCIT), Pune

Date: 20 September 2026

Abstract

Card fraud is rare but costly, which makes it a hard problem for standard classifiers: in the public Kaggle dataset used here only 0.173% of 284,807 transactions are fraudulent, so a model that always predicts “legitimate” is 99.83% accurate and useless. This report builds and compares two approaches on that dataset: a LightGBM gradient-boosted model tuned with Optuna and trained with SMOTE and class weighting, and a PyTorch feed-forward network trained with a class-weighted loss. Both were evaluated on a stratified hold-out set of 56,962 transactions (98 fraud cases) using precision-recall based metrics. LightGBM performed best overall (PR-AUC 0.8758, F1 0.8038), catching 84 of 98 frauds with 27 false alarms. The neural network caught more (90 of 98) but raised 1,592 false alarms (PR-AUC 0.7053). SHAP analysis showed that a small group of components, led by V14, drives the predictions. The report also documents methodological limitations, chiefly the use of the test set for early stopping, and proposes next steps.

Keywords: fraud detection, class imbalance, SMOTE, LightGBM, Optuna, neural network, PR-AUC, SHAP

1. Introduction

1.1 Background and problem

Fraudulent transactions make up a tiny share of payment traffic but cause large financial and reputational damage. The modelling challenge is not accuracy. It is finding as much fraud as possible (high recall) while keeping false alarms low enough for an operations team to review (high precision). Extreme class imbalance affects training, evaluation and the choice of metric alike.

1.2 Objectives

- Understand how fraudulent and legitimate transactions differ in the available data.
- Handle class imbalance without leaking information from the test data.
- Build and tune a tree-based model and a neural network, and compare them fairly.
- Explain individual and overall model decisions so that flagged transactions can be justified.
- Identify weaknesses in the evaluation and recommend improvements.

1.3 Scope

The work covers the full pipeline from exploratory analysis to explainability on a single public dataset. It does not cover deployment, real-time serving or cost-sensitive decision-making, which are discussed only as future work.

2. Dataset

The data is the Kaggle *Credit Card Fraud Detection* dataset, originally collected by Worldline and the Machine Learning Group of ULB from European cardholders over two days [1].

Item	Detail
Size	284,807 transactions, 31 columns, no missing values
Target	Class: 1 = fraud (492 rows), 0 = legitimate (284,315 rows)
Features	V1 to V28: anonymised principal components. Time: seconds since the first transaction. Amount: transaction value
Imbalance	Fraud rate 0.1727%, about 578 legitimate transactions per fraud

Because the components V1 to V28 are anonymised outputs of PCA, their real-world meaning is not available. This limits how far the results can be interpreted in business terms.

3. Methodology

3.1 Pipeline overview

Stage	Technique
Exploration	Class balance, amount and time distributions, correlation analysis
Preprocessing	RobustScaler on Amount and Time
Validation	Stratified 80/20 hold-out split; 5-fold stratified cross-validation
Imbalance handling	SMOTE (training data only); class weighting
Model A	LightGBM with Optuna hyperparameter search
Model B	PyTorch feed-forward network (FraudNet)
Evaluation	PR-AUC, ROC-AUC, F1, recall, precision, false-negative rate, confusion matrix
Explainability	SHAP (global and local)

All random seeds were fixed at 42 (Python, NumPy, PyTorch, the data split, SMOTE and Optuna) for reproducibility. Code was run in Google Colab, with the neural network trained on a CUDA GPU. Main libraries: scikit-learn [13], imbalanced-learn [14], LightGBM [3], Optuna [4], PyTorch [12] and SHAP [6].

3.2 Preprocessing

V1 to V28 were already standardised by PCA, so only Amount and Time were scaled. RobustScaler was used instead of standard scaling because it relies on the median and inter-quartile range, so unusually large amounts (often the fraud cases) do not distort the scaling:

$$x' = (x - \text{median}(x)) / (Q75(x) - Q25(x))$$

3.3 Validation design

The data was split 80/20 with stratification, giving 227,845 training rows (394 fraud) and 56,962 test rows (98 fraud), each with a fraud rate close to 0.172%. Stratification matters here: with so few positives, a plain random split could produce folds with very different fraud rates. Five-fold stratified cross-validation on the training set was used for hyperparameter tuning.

3.4 Handling class imbalance

SMOTE [2] creates synthetic minority examples by interpolating between a real fraud case and one of its nearest fraud neighbours. It was configured to raise the fraud class to 10% of the legitimate class, taking the training set from 394 to 22,745 fraud rows (9.09% of 250,196 rows). SMOTE was applied only to training data, inside each cross-validation fold, so that synthetic points never appear in validation or test data. **Class weighting** was the second lever: LightGBM used *scale_pos_weight*, and the neural network used a positive-class weight of 577.1 in its loss instead of SMOTE, so that every network input remains a real transaction.

3.5 Model A: LightGBM with Optuna

LightGBM [3] is a gradient-boosted decision-tree method that captures non-linear interactions natively and is insensitive to feature scaling. Hyperparameters were tuned with Optuna [4] using the Tree-structured Parzen Estimator sampler [5]. The search ran for 30 trials, each scoring the mean PR-AUC across five folds (SMOTE inside each fold, early stopping after 50 rounds). The search space covered learning rate, tree size and depth, minimum child samples, subsampling, regularisation and *scale_pos_weight*.

3.6 Model B: FraudNet (PyTorch)

FraudNet is a compact feed-forward network: 30 inputs, hidden layers of 128, 64 and 32 units, and a single output logit. Each hidden layer is followed by batch normalisation [9], ReLU and 30% dropout [10].

Setting	Choice
Loss	Binary cross-entropy with logits, positive-class weight 577.1
Optimiser	AdamW [11], learning rate 1e-3, weight decay 1e-5, batch size 2,048
Scheduling	ReduceLROnPlateau (factor 0.5, patience 3)
Stopping	Early stopping on validation loss (patience 8, maximum 60 epochs)
Data	Training set split 85/15 (stratified) into train and validation

3.7 Evaluation metrics

Accuracy was deliberately not used. Precision-recall based metrics were chosen because ROC curves can look overly optimistic when negatives vastly outnumber positives [8]. With TP, FP and FN denoting true positives, false positives and false negatives:

Metric	Definition	Why it matters here
Precision	$TP / (TP + FP)$	Share of alerts that are real fraud (analyst workload)
Recall	$TP / (TP + FN)$	Share of fraud caught (money not lost)
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Single balance of the two
False-negative rate	$FN / (FN + TP)$	Missed fraud; equals 1 minus recall
PR-AUC	Area under the precision-recall curve (average precision)	Headline, threshold-free metric
ROC-AUC	Area under the ROC curve	Reported for completeness

Threshold-based metrics used a decision threshold of 0.5 for both models.

4. Results

4.1 Exploratory findings

Class imbalance. Figure 1 uses a log scale because the fraud bar would otherwise be invisible.

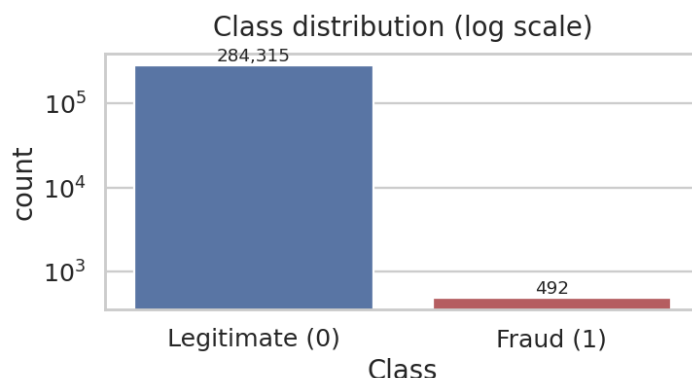


Figure 1. Class distribution (log scale): 284,315 legitimate vs 492 fraud.

Amount and time. Fraud does not look like a smaller copy of legitimate activity (Figure 2). Fraud amounts spike at roughly €1 and cluster again near €100, while legitimate amounts spread broadly between about €10 and €200. Legitimate volume follows a smooth two-day rhythm, whereas fraud is spiky, with pronounced peaks around 40,000 s and 95,000 s.

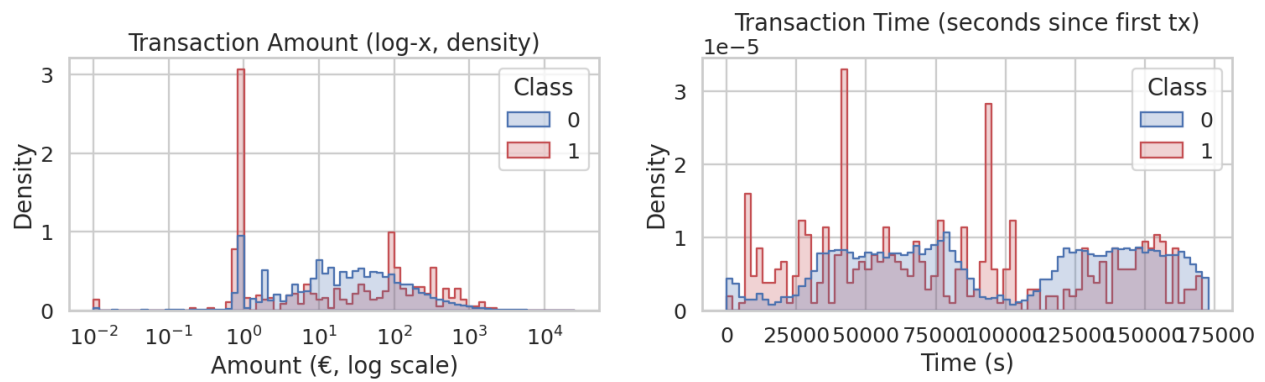


Figure 2. Amount (log x-axis) and Time distributions by class, shown as densities.

Correlations. No single component is strongly correlated with fraud (Table 1, maximum about 0.33), a first hint that linear models would struggle.

Table 1. Ten strongest absolute correlations with Class

Rank	Feature	[corr]	Rank	Feature	[corr]
1	V17	0.326	6	V3	0.193
2	V14	0.303	7	V7	0.187
3	V12	0.261	8	V11	0.155
4	V10	0.217	9	V4	0.133
5	V16	0.197	10	V18	0.111

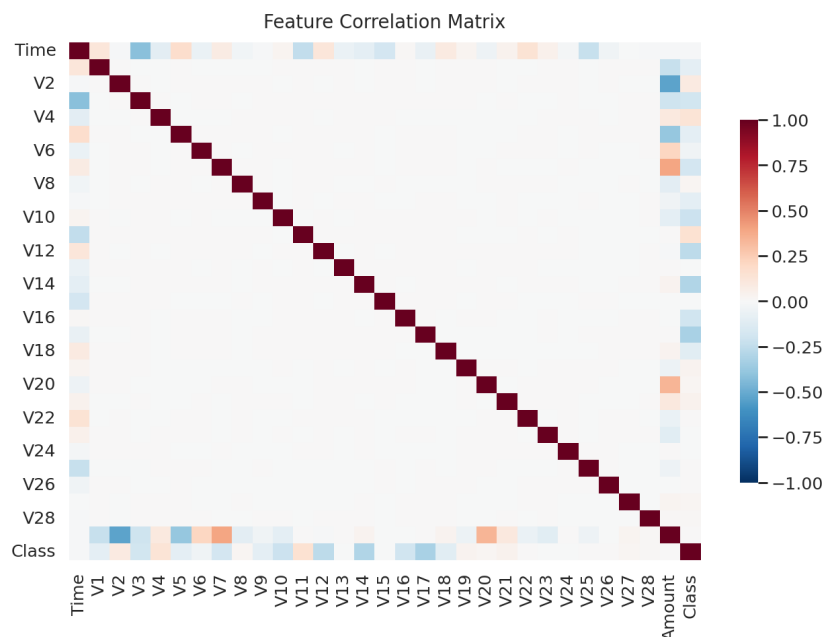


Figure 3. Feature correlation matrix. The near-diagonal pattern confirms the PCA components are orthogonal.

4.2 LightGBM

The best cross-validated PR-AUC from the 30 Optuna trials was **0.8550**, using the parameters in Table 2.

Table 2. Selected LightGBM hyperparameters

Parameter	Value	Parameter	Value
learning_rate	0.1416	subsample	0.680
num_leaves	252	colsample_bytree	0.620
max_depth	10	reg_alpha	0.0103
min_child_samples	78	reg_lambda	0.0624
scale_pos_weight	360.5	n_estimators (max)	2000, early-stopped

Retrained on the full resampled training set, the model produced the hold-out results below (PR-AUC 0.8758, ROC-AUC 0.9780): 84 frauds caught, 14 missed and 27 legitimate transactions wrongly flagged. The small gap between cross-validated and hold-out PR-AUC suggests the tuning did not badly overfit the folds.

Class	Precision	Recall	F1	Support
Legitimate	0.9998	0.9995	0.9996	56,864
Fraud	0.7568	0.8571	0.8038	98

4.3 FraudNet

Training stopped at **epoch 13**. Validation loss was lowest at epoch 5 (0.3800) while training loss kept falling from 0.77 to about 0.19, the typical overfitting pattern (Figure 4), so the epoch-5 weights were restored. On the hold-out set the network reached PR-AUC 0.7053 and ROC-AUC 0.9824, catching 90 frauds and missing 8, but with 1,592 false alarms.

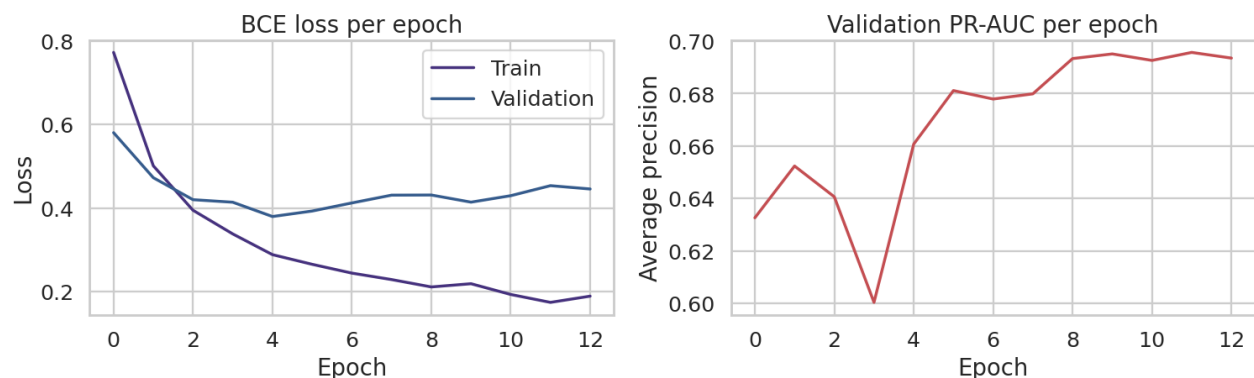


Figure 4. FraudNet training curves: BCE loss (left) and validation PR-AUC (right) per epoch.

Class	Precision	Recall	F1	Support
Legitimate	0.9999	0.9720	0.9857	56,864
Fraud	0.0535	0.9184	0.1011	98

4.4 Model comparison

Table 3. Hold-out performance at a 0.5 threshold (56,962 transactions, 98 fraud)

Metric	LightGBM	FraudNet (DL)
PR-AUC (headline)	0.8758	0.7053
ROC-AUC	0.9780	0.9824
F1 (fraud)	0.8038	0.1011
Recall (fraud)	0.8571	0.9184
Precision (fraud)	0.7568	0.0535
False-negative rate	0.1429	0.0816
True positives / False positives	84 / 27	90 / 1,592
False negatives / True negatives	14 / 56,837	8 / 55,272
Transactions flagged	111 (0.19% of traffic)	1,682 (2.95% of traffic)

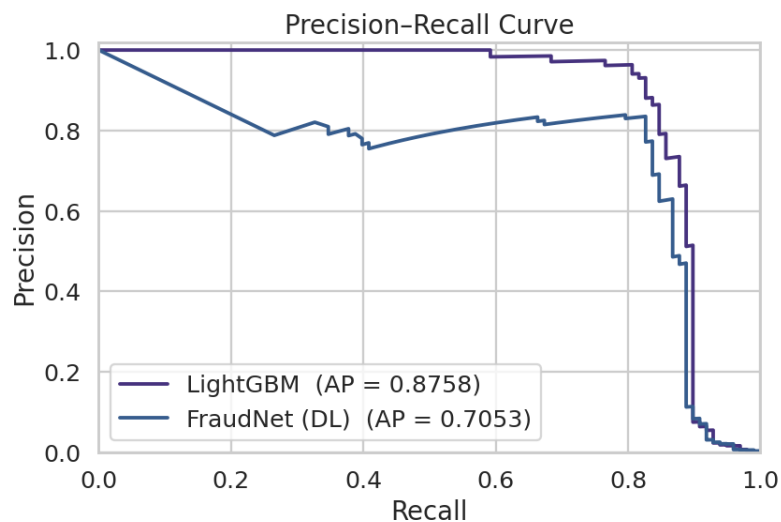


Figure 5. Precision-recall curves for both models on the hold-out set.

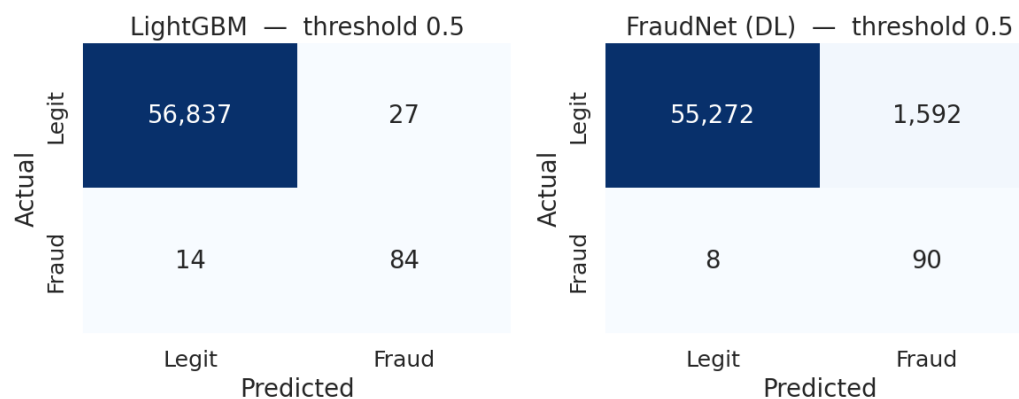


Figure 6. Confusion matrices at a 0.5 decision threshold.

4.5 Explainability (SHAP)

SHAP [6] attributes each prediction to individual features, and its TreeSHAP variant computes this exactly for tree models [7]. It was run on the full hold-out set for LightGBM (Figure 7).

- **V14 dominates.** Low V14 values push scores strongly toward fraud.
- **V4, V8, V1, V12 and V3 follow.** High V4 raises fraud risk, while low V12 and low V3 do the same.
- **Correlation is not importance.** V17 had the strongest raw correlation with Class (Table 1) but is absent from the top 20 SHAP features. It probably overlaps with V14, V12 and V10, which the trees use instead.
- **Time and Amount matter less than expected:** Time_scaled sits mid-table and Amount_scaled near the bottom of the top 20.

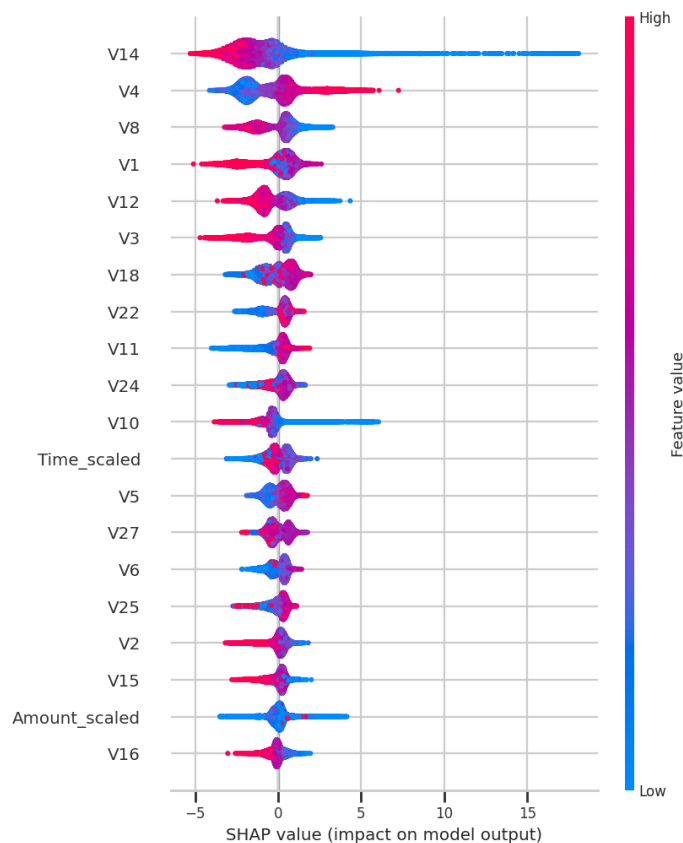


Figure 7. SHAP summary plot for LightGBM (top 20 features). Red = high feature value, blue = low.

For a single decision, the highest-scoring true fraud (test row 50,636, predicted probability 1.0000) was examined (Figure 8). Its final score of 22.04 sits far above the baseline of -15.44, and V14 alone contributes +16.15, followed by V10 (+4.94), V4 (+4.82) and V12 (+2.93). This is the kind of per-transaction reasoning an analyst or auditor needs when a card is blocked.

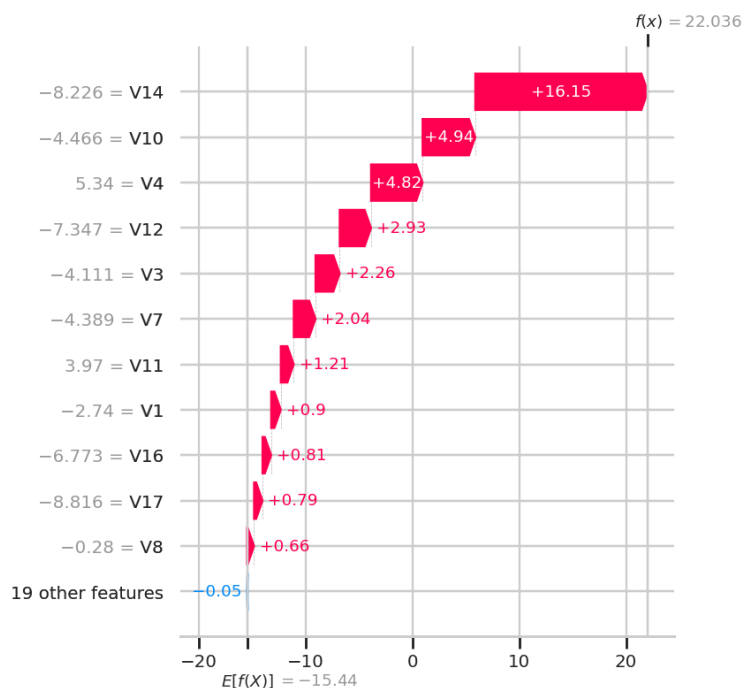


Figure 8. SHAP waterfall for the highest-confidence fraud in the hold-out set.

5. Discussion

Which model is better? On the metrics that matter for fraud, LightGBM is stronger. FraudNet's six extra catches (90 vs 84) came with 1,565 extra false alarms, roughly 260 additional reviews per extra fraud found, and it would flag about 3% of all traffic against 0.19% for LightGBM. The PR-AUC gap (0.876 vs 0.705) shows LightGBM separates the classes better in the region that matters.

Why ROC-AUC misleads. FraudNet's ROC-AUC (0.9824) is marginally higher than LightGBM's (0.9780), yet its precision is only 5.4%. Because legitimate transactions vastly outnumber fraud, even a modest false-positive rate produces a large number of false alarms, and ROC-AUC barely registers it. This is a practical illustration of why PR-based metrics were preferred [8].

A possible threshold effect. FraudNet's weak precision is partly a product of the fixed 0.5 threshold combined with a very large class weight, which pushes predicted probabilities upward. Its ranking ability (ROC-AUC) is competitive, so a tuned threshold might narrow the precision gap. This was not tested, so the conclusion above is limited to the 0.5 operating point.

Interpretability. The SHAP results are consistent across the global and local views: V14 is the dominant signal. Since the components are anonymised, however, they explain *what* the model uses, not *why* in business terms.

6. Limitations

- **Test-set early stopping (LightGBM).** The final LightGBM fit uses the test set as its early-stopping evaluation set, so the number of trees was chosen using test data. Hold-out metrics are therefore slightly optimistic.
- **Scaler fitted before the split.** RobustScaler was fitted on the whole dataset. The effect on a median and IQR is tiny, but strictly it should be fitted on training data only.
- **Very few positives.** With 98 fraud cases in the test set, each miss moves recall by about one percentage point. Results come from a single split with no confidence intervals.
- **Random split.** Real systems predict the future from the past. A chronological split would give a more honest estimate, especially given the bursty fraud timing in Figure 2.
- **Fixed threshold.** Both models were scored at 0.5, which disadvantages FraudNet in particular.
- **FraudNet checkpoint choice.** Weights were selected on validation loss (epoch 5, validation PR-AUC about 0.66), although validation PR-AUC kept improving to about 0.70 later. Selecting on PR-AUC might give a different result.
- **Single dataset.** The data covers two days of European transactions, so the findings may not generalise to other regions, periods or fraud patterns.

7. Conclusion and recommendations

The study shows that with careful handling of imbalance (SMOTE inside the folds plus class weighting), a tuned gradient-boosted model can detect roughly 86% of fraud at about 76% precision on this dataset, clearly outperforming a comparable neural network at the default threshold. LightGBM is recommended as the primary model, with FraudNet kept as a benchmark and possible ensemble member. SHAP makes the model's decisions auditable, both overall and per transaction.

Recommended next steps

- Fix the evaluation hygiene: separate validation data for early stopping, scaler fitted on training data only, then re-report the hold-out metrics.
- Choose the threshold from a cost matrix (a missed fraud costs roughly the transaction value; a false alarm costs analyst time), and re-test FraudNet at its own tuned threshold.
- Validate with a chronological split and check stability across periods.
- Test stacking of the two models to see whether their errors are complementary.
- Plan for deployment: low-latency model export and drift monitoring (for example Population Stability Index) on the top SHAP features such as V14, V4, V8 and V12.
- Repeat the study on a dataset with real categorical and identity features (such as IEEE-CIS) for more meaningful explanations.

References

- [1] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *2015 IEEE Symposium Series on Computational Intelligence*, 159-166.
- [2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [3] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146-3154.
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD Conference*, 2623-2631.
- [5] Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems*, 24, 2546-2554.
- [6] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765-4774.
- [7] Lundberg, S. M., Erion, G., Chen, H., et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56-67.
- [8] Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.
- [9] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning*, 448-456.
- [10] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958.
- [11] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.
- [12] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024-8035.
- [13] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [14] Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1-5.

Appendix: reproducibility

Item	Detail
Random seed	42 for Python, NumPy, PyTorch, train/test split, SMOTE and Optuna
Environment	Google Colab, PyTorch on CUDA GPU
Pinned packages	lightgbm 4.5.0, optuna 3.6.1, shap 0.46.0, imbalanced-learn 0.12.4, plotly 5.24.1
Data	Kaggle <i>mlg-ulb/creditcardfraud</i> (<i>creditcard.csv</i> , 144 MB)
Source notebook	<code>fraud_detection_end_to_end.ipynb</code>